Advanced Docker:

1. Image layers and union file systems

A Docker image is built from **read-only layers**. Each instruction like RUN, COPY, or ADD creates a new layer.
When a container starts, Docker adds a **thin writable layer** on top.
Why it matters:

- Faster builds through layer caching
- Smaller pushes/pulls when layers are reused
- Better Dockerfile design for performance

Example idea:

- Put rarely changing steps first
- Put frequently changing app code later

2. Multi-stage builds

Multi-stage builds let you use one stage to **build** the app and another to **run** it. This keeps the final image smaller and more secure.

```
FROM golang:1.24 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

FROM alpine:3.20
WORKDIR /app
COPY --from=builder /app/myapp .
CMD ["./myapp"]
```

Benefit:

- Final image excludes compilers, build tools, and source clutter

3. Build cache optimization

Docker reuses cached layers when inputs have not changed.
Common optimization:

```
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
```

This works well because dependency install only reruns when package files change.

4. .dockerignore

This file prevents unnecessary files from being sent to the Docker build context.
Typical entries:

```
node_modules
.git
dist
*.log
```

.env
Why it matters:

- Faster builds
- Smaller context
- Less risk of secrets being copied

5. Volumes, bind mounts, and tmpfs

These are different storage mechanisms.
Bind mount

- Maps a host path into the container
- Great for development

docker run -v $(pwd):/app myimage
Volume

- Managed by Docker
- Better for persistent production data

docker volume create appdata
docker run -v appdata:/var/lib/mysql mysql
tmpfs

- In-memory storage
- Useful for temporary sensitive data

6. Container networking

Docker has multiple network drivers.

Bridge

Default for containers on one host.

Host

Container shares host network stack.

None

No networking.

Overlay

Used across multiple hosts, typically in Swarm.
Useful commands:
docker network ls
docker network create mynet
docker run --network mynet nginx

7. DNS and service discovery

Containers on the same user-defined network can talk using **container/service names**.
Example:

- frontend can connect to db:5432
- No need to hardcode IP addresses

This is especially important in Docker Compose.

8. Entrypoint vs CMD

These are often confused.

CMD

- Default command arguments
- Easier to override

ENTRYPOINT

- Defines the main executable
- Makes container act like a fixed program

Example:

ENTRYPOINT ["python"]
CMD ["app.py"]
This runs python app.py by default.

9. PID 1 and signal handling

Inside a container, the main process becomes **PID 1**.
PID 1 behaves differently, especially with signals and zombie process reaping.
Problems:

- App may not shut down cleanly
- Child processes may not be reaped

Common fix:

- Use exec form
- Use a minimal init like tini

ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "server.js"]

10. Healthchecks

A container running does not always mean the app is healthy.
HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1
This helps orchestration tools know when the app is actually ready or broken.

11. Resource limits

Docker lets you control CPU and memory.
Examples:
docker run --memory=512m --cpus=1.5 myimage
Why important:

- Prevent one container from consuming the whole host
- Simulate production constraints

12. Security hardening

Important advanced practice areas:

Run as non-root

RUN adduser -D appuser
USER appuser

Drop Linux capabilities

docker run --cap-drop ALL myimage

Read-only filesystem
docker run --read-only myimage
Scan images
Check for vulnerabilities in base images and dependencies.

13. Secrets management
Do not bake secrets into images.
Bad:
ENV DB_PASSWORD=secret123
Better:
- Environment injection at runtime
- Docker secrets in Swarm
- External secret managers

14. Logging drivers
Containers usually write logs to stdout/stderr, and Docker handles collection.
Useful logging drivers:
- json-file
- syslog
- journald
- fluentd
- gelf

This matters in production when integrating centralized logging.

15. Docker Compose advanced use
Compose is more than starting two containers.
Advanced features:
- Multiple services
- Named volumes
- Custom networks
- Environment file support
- Profiles
- Dependency coordination
- Override files for dev/prod

Example:
```
services:
  app:
    build: .
    depends_on:
      - db
    networks:
      - backend

  db:
    image: postgres:16
    volumes:
```

```
      - pgdata:/var/lib/postgresql/data
    networks:
      - backend

volumes:
  pgdata:

networks:
  backend:
```

16. Orchestration concepts

Docker alone manages containers on one machine.
For many containers across hosts, you need orchestration.
Key ideas:

- Scheduling
- Auto-restart
- Rolling updates
- Service discovery
- Scaling
- Self-healing

Historically with Docker:

- **Docker Swarm**
- More commonly now: **Kubernetes**

17. Image registries and distribution

Images are stored in registries:

- Docker Hub
- Private registries
- Cloud registries

Advanced concerns:

- Tag strategy
- Immutable versions
- Registry auth
- Content trust
- Pull policies

Good practice:

- Avoid relying only on latest
- Use versioned tags like 1.4.2

18. Rootless Docker

Rootless Docker runs Docker without full root privileges.
Benefits:

- Better isolation
- Reduced host risk

Tradeoff:

- Some networking/storage features may behave differently

19. BuildKit

BuildKit is Docker's newer build engine.
Advantages:

- Faster builds
- Better caching
- Secret mounts during build
- SSH forwarding
- Parallel execution

Example:
syntax=docker/dockerfile:1
RUN --mount=type=secret,id=mysecret cat /run/secrets/mysecret

20. Container lifecycle and restart policies

Useful restart options:
docker run --restart=always myimage
docker run --restart=on-failure myimage
Lifecycle states matter when debugging:

- created
- running
- paused
- exited
- restarting

21. Debugging containers

Advanced debugging commands:
docker logs <container>
docker exec -it <container> sh
docker inspect <container>
docker stats
docker events
What they help with:

- Startup failures
- Environment issues
- Port bindings
- Resource pressure
- Network problems

22. Distroless and minimal images

Instead of full OS images, use very small runtime images.
Examples:

- Alpine
- Distroless
- Scratch

Benefits:

- Smaller attack surface

- Faster pulls
- Smaller footprint

Tradeoff:

- Harder to debug because common shell tools may be missing

23. Copy-on-write performance implications

Container filesystems use copy-on-write.
This is efficient, but write-heavy workloads may perform worse than expected.
Practical rule:

- Use volumes for databases and heavy write workloads

24. Platform and architecture awareness

Docker supports multiple architectures:

- amd64
- arm64

Useful when building for cloud servers, Apple Silicon, or edge devices.
Example:
docker buildx build --platform linux/amd64,linux/arm64 .

25. Common advanced interview topics

These are asked a lot:

- Difference between image and container
- Difference between CMD and ENTRYPOINT
- Multi-stage build use case
- Bind mount vs volume
- Why run containers as non-root
- How Docker networking works
- What happens when PID 1 ignores signals
- How layer caching affects builds

Best practices summary

- Keep images small
- Use multi-stage builds
- Use .dockerignore
- Prefer non-root users
- Avoid storing secrets in images
- Use healthchecks
- Pin versions
- Use volumes for persistent data
- Understand signals and PID 1
- Use BuildKit and caching effectively